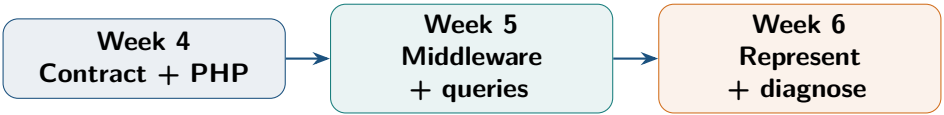
420-511-VA Web Services

Week 6
Content negotiation and Resource Representation Design

With JSON, XML, and YAML

Pravish Sainath

From Collections to Dependable Responses



Alex has a new integration

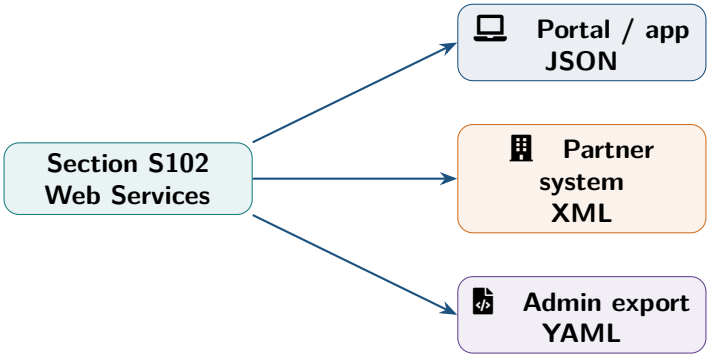
The portal uses JSON. A partner needs XML. An administrator wants YAML. If a request fails, every client needs an understandable outcome and the operator needs a useful log.

Takeaway: Keep the resource and business rules stable; make representation and failure policies explicit.

What You Will Be Able to Do

1. Design one public data model and map it to JSON, XML, and YAML.
2. Distinguish request `Content-Type` from response preferences in `Accept`.
3. Reason about quality values, `406`, `415`, caching, and variants.
4. Implement serializers and error boundaries that return PSR-7 responses.
5. Create safe Problem Details responses and structured, correlated logs.
6. Test the messages and operational evidence, including failure paths.

One Resource, Several Consumers



Takeaway: These are synthetic consumers, not claims about the college's real infrastructure.

The Week 6 Teaching Contract

Area	Explicit choice
Existing data	Keep the Week 5 S101–S108 sections and page semantics.
Reads	`/v2/sections`; add illustrative member route `/v2/sections/{id}`.
Success formats	`application/json`, `application/xml`, `application/yaml`. JSON remains the default.
Writes	Existing enrollment writes still accept JSON only; XML/YAML output does not imply XML/YAML input.
Errors / caching	Fixed `application/problem+json` fallback; retain `Cache-Control: no-store`.

New teaching extension, not a deployed service. PHP 8.2+ concepts; PSR-7/15; OpenAPI 3.1.2; Symfony Yaml 7.4 and Monolog 3 examples.

1. Designing Resource Representations

Preserve meaning before choosing a serialization format

Resource, Representation, and Storage Are Different



Resource state

The section has a term, capacity, enrolled count, and availability. The database holds authoritative state.

A representation

The JSON document is a transferable view of that state at a point in time. XML can describe the same resource.

Takeaway: Changing the format does not create a new section or reserve a seat.

RFC 9110: HTTP semantics

Start with the Same Week 5 Data

Property	Value	Meaning
`id`	`S102`	Stable public identifier; a string.
`courseCode`	`420-511`	Identifies the course; not a numeric quantity.
`title`	Web Services	Human-readable title from the related course.
`term / state`	`F26 / open`	Documented values; not free-form display messages.
`capacity / availableSeats`	`30 / 4`	Integer quantities; availability is calculated.

Takeaway: S102 has capacity 30 and four available seats in the fixture; do not change facts between formats.

Design the Public View Deliberately

Decision	Useful contract rule
Names	Use `availableSeats` publicly; storage can keep `seats_taken`.
Types	Keep IDs, quantities, booleans, and dates consistent across responses.
Exposure	Allowlist public properties. Never dump a complete ORM object or database row.
Relationships	Choose an identifier, a link, or a bounded embedded summary.
Evolution	Document optional fields and client tolerance; do not silently change types.

Takeaway: A serializer cannot decide which personal or internal data a caller may see.

JSON Schema: objects | OWASP: REST security

Filtering Is Not Formatting

Week 5: which rows?

```
GET /v2/sections?term=F26&state=open
    &sort=-availableSeats,id&page=2&perPage=2

Items: S108, S107
```

Filters, ordering, and pagination determine the result.

Week 6: how to express it?

```
Accept: application/xml

Same selected IDs, same order,
same page meaning, different bytes.
```

The serializer must not re-filter or reorder the collection.

Takeaway: Field selection changes the view; media-type negotiation changes the representation format.

JSON: A Compact Data Model


```
{
  "id": "S102",
  "courseCode": "420-511",
  "title": "Web Services",
  "term": "F26",
  "state": "open",
  "capacity": 30,
  "availableSeats": 4
}
```

- Objects contain named properties; arrays preserve sequence.
- Numbers, strings, booleans, and `null` are distinct values.
- Use `application/json` and UTF-8 for interoperable HTTP exchange.

A JSON object's property order is not the resource's ordering contract; array order matters.

RFC 8259: JSON

JSON: Small Mistakes, Different Meanings

Example	Interpretation / issue
`"availableSeats": 4`	Number; can satisfy an integer schema.
`"availableSeats": "4"`	String; not interchangeable with the number.
`"items": []` vs `"items": {}`	Empty list versus empty object.
`null` vs `"null"` vs omitted	Explicit empty value, literal text, or no property.
Comments, trailing commas, `NaN`	Not valid standard JSON. Duplicate property names harm interoperability.

Takeaway: Parse successfully, then validate the shape and meaning. Valid JSON alone is not a valid API request.

RFC 8259: JSON | JSON Schema: objects

PHP Arrays Can Change the JSON Shape


```php
$rows = [0 => ['id' => 'S101'], 2 => ['id' => 'S103']];
echo json_encode($rows); // {"0":{"id":"S101"},"2":{"id":"S103"}}

echo json_encode(array_values($rows)); // [{"id":"S101"},{"id":"S103"}]

echo json_encode([]);          // []
echo json_encode(new stdClass()); // {}
```

Takeaway: After filtering a PHP list, reindex it if the contract promises a JSON array. Do not use JSON_FORCE_OBJECT indiscriminately.

Encoding examples are separate calls for illustration, not one combined HTTP response.

PHP: json_encode

XML: An Explicit Document Structure


```xml
<?xml version="1.0" encoding="UTF-8"?>
<section>
  <id>S102</id>
  <courseCode>420-511</courseCode>
  <title>Web Services</title>
  <term>F26</term>
  <state>open</state>
  <capacity>30</capacity>
  <availableSeats>4</availableSeats>
</section>
```

- One root element; matching, case-sensitive element names.
- Our contract uses child elements for the public properties.
- Send `application/xml; charset=utf-8` and matching UTF-8 bytes.

Element text is not automatically an integer. A mapping or XML Schema supplies that interpretation.

W3C XML 1.0 | RFC 7303: XML media types

XML Structure Is a Design Choice


```xml
<section id="S102">
  <title>Web Services</title>
</section>

<section xmlns="https://api.example.edu/ns">
  <id>S102</id>
</section>
```

- **Elements versus attributes:** both exist; choose one mapping and keep it stable.
- **Namespaces:** identify names unambiguously across vocabularies; a namespace URI is not necessarily a download URL.
- **This deck:** simple unqualified section elements; more elaborate XML consumers may need XSD.

Takeaway: Do not assume a generic JSON-to-XML conversion produces the XML contract a partner expects.

W3C XML 1.0 | OpenAPI 3.1.2

Collections and Null Need an XML Mapping

JSON / YAML meaning

```
{
  "items": [],
  "links": {"next": null}
}
```

One explicit XML mapping

```
<sectionPage
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <items />
  <links><next xsi:nil="true" /></links>
</sectionPage>
```

- The `<items>` wrapper contains ordered `<section>` child elements.
- Here `<items />` means an empty list; `xsi:nil="true"` means null by contract.

Takeaway: An absent element, an empty element, and a nil element need not mean the same thing.

W3C XML 1.0 | OpenAPI 3.1.2

YAML: Readable Structured Data


```yaml
id: S102
courseCode: '420-511'
title: Web Services
term: F26
state: open
capacity: 30
availableSeats: 4
```

- Mappings use `key: value`; sequences use `-` entries.
- Indent with spaces, not tabs. Quote strings when their type is ambiguous.
- Use the registered `application/yaml` media type; `text/yaml` is a legacy alias.

Takeaway: A YAML OpenAPI file describes an API; a YAML response represents a resource. Those are different uses.

YAML 1.2.2 | RFC 9512: application/yaml

YAML: Type Surprises and Powerful Features


```
studentCode: '00142'           # keep leading zeros as text
publishedAt: '2026-10-02T14:00:00Z'  # explicit string
active: true                   # boolean
note: null                     # null, not the text "null"
items: []                      # empty sequence
```

- Scalar resolution can differ by YAML version, schema, and parser. Test the chosen implementation.
- Aliases, custom tags, and object construction add complexity beyond a simple JSON-like model.
- For this API, emit simple mappings, sequences, and scalar values; do not deserialize arbitrary objects.

Takeaway: Human-friendly syntax does not remove the need for a schema or safe parser settings.

RFC 9512: application/yaml | Symfony 7.4: Yaml component

Choose Formats for Consumers, Not Fashion

Format	Good fit	Trade-off
JSON	Browser/mobile API data; familiar tooling.	No comments; precision and array/object distinctions matter.
XML	Established document contracts and integration partners.	Explicit mapping, namespaces, escaping, and parser hardening.
YAML	Readable exports and configuration files.	Implicit types and advanced features require a constrained profile.

Store example: JSON for an app, XML for a partner, YAML for an export; one product identity.

Takeaway: Supporting three response formats does not oblige the API to accept all three as input.

RFC 8259: JSON | RFC 7303: XML media types | RFC 9512: application/yaml

Preserve Meaning Across Every Format

Concern	Deliberate rule
Collections	Keep the same IDs, order, totals, and navigation meaning in each variant.
Money	Example: `priceCents: 15000` plus `currency: CAD`; define units explicitly.
Large identifiers	Use strings when client number precision cannot represent the identifier exactly.
Time	Use an explicit timezone and documented timestamp shape; avoid locale-formatted dates.
Null and missing	Keep the distinction explicit in the JSON/YAML schema and XML mapping.

Takeaway: Equivalent meaning does not imply byte equality. Test decoded views, not just string equality.

RFC 8259: JSON | Symfony 7.4: Yaml component

Escaping Is the Serializer's Job

A perfectly ordinary course title

```
Data & Web <Basics>
```

These characters are data, not markup.

An XML encoder writes

```
<title>Data &amp;amp; Web &amp;lt;Basics&amp;gt;</title>
```

A parser reconstructs the original text.

- Use `json_encode`, `XMLWriter`, and a maintained YAML dumper.
- Do not concatenate user text into JSON, XML tags, or log lines.

Takeaway: Escaping for one format does not sanitize data for every other context.

PHP: XMLWriter | PHP: json_encode

Representation Checkpoint

Three questions

A section is returned as JSON and XML. The XML says `availableSeats=5` while JSON says `4`. Is this just formatting?

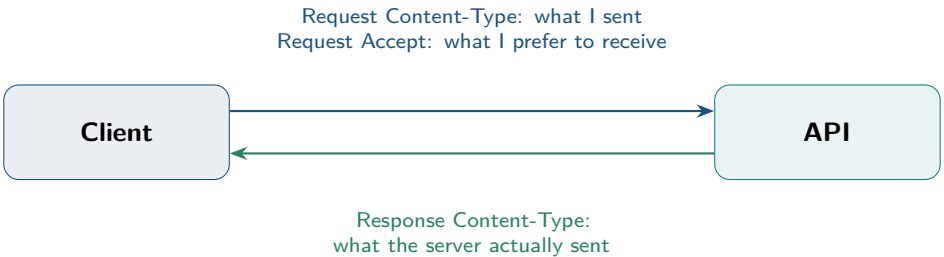
- **No:** the views disagree about the data, unless they were explicitly read at different times.
- An empty result stays an empty collection in every format.
- Changing from `4` to `"4"` is a type change, not harmless pretty-printing.

Takeaway: Review identity, structure, types, and semantics before adjusting indentation.

2. Content Negotiation

The client states preferences; the server selects an available representation

Accept and Content-Type Point in Different Directions



Takeaway: Accept does not identify the request body. Content-Type does not ask the server to return that format.

RFC 9110, 12.5.1: Accept | RFC 9110: HTTP semantics

Sending JSON and Receiving XML Is Possible


```
POST /exports HTTP/1.1
Content-Type: application/json
Accept: application/xml, application/problem+json

{"sectionId":"S102"}
```

```
HTTP/1.1 202 Accepted
Content-Type: application/xml; charset=utf-8
Location: /exports/job-7
Vary: Accept

<job><id>job-7</id><state>queued</state></job>
```

Takeaway: Request decoding and response encoding are independent decisions.

Illustrative export API, separate from the existing JSON-only enrollment write contract. A 202 response does not mean the export has finished.

RFC 9110: HTTP semantics

Start with an Explicit Format Preference


```
GET /v2/sections/S102 HTTP/1.1
Accept: application/xml

HTTP/1.1 200 OK
Content-Type: application/xml; charset=utf-8
Vary: Accept
Cache-Control: no-store

<section>...</section>
```

Input preference	Our choice
No `Accept` field	Treat all formats as acceptable; default to JSON.
`Accept: */*`	JSON wins our documented server preference order.
`Accept: application/yaml`	Return YAML if the target supports it.

Ellipsis shortens only this displayed body. Actual XML must be complete and well-formed.

RFC 9110, 12.5.1: Accept

Quality Values Express Relative Preference


```
Accept: application/xml;q=0.9, application/json;q=0.6,
        application/yaml;q=0.2
```

Available candidate	Effective quality	Result
`application/xml`	`0.9`	Preferred supported representation.
`application/json`	`0.6`	Acceptable alternative.
`application/yaml`	`0.2`	Lower preference.

Takeaway: q defaults to 1; q=0 excludes a match. Values range from 0 through 1, with at most three decimal places.

Header wrapping is for display; clients should send valid field syntax. q is not a probability or server-performance score.

RFC 9110, 12.5.1: Accept

Specificity Comes Before Comparing Candidates


```
Accept: application/*;q=0.8, application/json;q=0,
        application/xml;q=0.4
```

Candidate	Most specific match	Quality
JSON	`application/json;q=0`	`0` – excluded.
XML	`application/xml;q=0.4`	`0.4`
YAML	`application/*;q=0.8`	`0.8` – selected.

Takeaway: A broad wildcard must not override an explicit exclusion of JSON.

Match the most specific applicable range for each candidate, then compare their effective qualities.

RFC 9110, 12.5.1: Accept

A Negotiation Algorithm You Can Explain

1. Start with the formats actually supported by this operation.
2. Parse the Accept ranges, parameters, and quality values.
3. For each candidate, find its most specific matching range.
4. Associate that range's quality; exclude candidates with q=0.
5. Choose the highest-quality candidate; resolve ties using a documented server preference.
6. If none is acceptable, apply the documented fallback or return 406.

Takeaway: Do not implement negotiation with a substring search such as contains("json").

HTTP permits ignoring an unsatisfied Accept preference. Our successful-response policy instead returns 406; see the separate error-format policy.

RFC 9110, 12.5.1: Accept

406 and 415 Diagnose Different Problems

⚠ 406 Not Acceptable

```
GET /v2/sections/S102
Accept: image/png

406 Not Acceptable
```

The server cannot offer a requested success representation under our policy.

⚠ 415 Unsupported Media Type

```
POST /v1/enrollments
Content-Type: application/xml

415 Unsupported Media Type
```

This operation accepts a JSON request body, not XML.

💡 **Takeaway:** 406 concerns response selection. 415 concerns unsupported request content, including unsupported coding when relevant.

RFC 9110: HTTP semantics | RFC 9110, 12.5.1: Accept

A Media Type Is Not Just a File Extension

- `application/json` and `application/problem+json` are distinct media types.
- The `+json` suffix identifies a JSON-based syntax; it does not make all such types interchangeable.
- An endpoint may use `?format=xml` or `.xml` instead of Accept, but that is a separate API convention.
- Do not invent an undocumented precedence between a format parameter and Accept.

Takeaway: Our Week 6 design selects by Accept. The Week 5 query parser still rejects unknown format parameters.

RFC 9110: HTTP semantics | RFC 9457: Problem Details

Format, Language, and Compression Are Separate

Request field	What it selects	Response evidence
`Accept`	Representation media type.	`Content-Type`
`Accept-Language`	Preferred human language.	`Content-Language` when used.
`Accept-Encoding`	Content coding such as gzip.	`Content-Encoding`

A practical combination

A phone can receive French-language JSON compressed with gzip. Compression changes the transmitted bytes; it does not turn JSON into XML.

Takeaway: We standardize UTF-8 rather than introducing a charset-negotiation feature.

RFC 9110: HTTP semantics

Two Ways to Choose a Representation

Proactive: server selects

- The client sends preferences with the first request.
- The server chooses an available variant.
- Our course API uses this model.

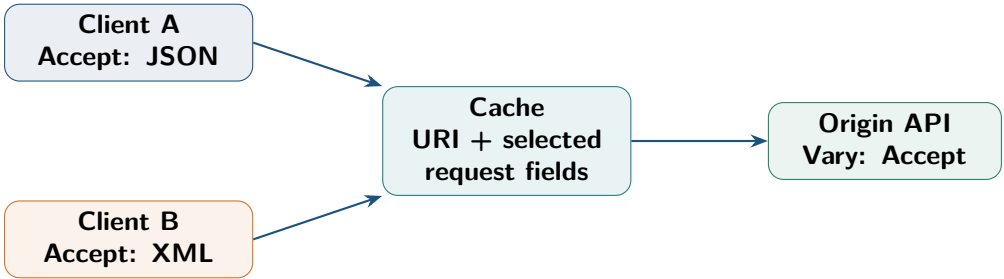
Reactive: client selects

- The response identifies alternatives.
- The client chooses a linked representation or export.
- Useful when a person wants to choose a downloadable report.

Takeaway: Neither model means duplicating the business rules for each format.

RFC 9110: HTTP semantics

Caches Must Distinguish Negotiated Variants



Takeaway: When selection depends on Accept, send Vary: Accept on those representations; merge with existing Vary values.

Vary is not permission to cache. Cache-Control and private access rules still apply. Our classroom responses retain no-store.

RFC 9111: caching | RFC 9110: HTTP semantics

Validators Belong to the Selected Representation


```
GET /catalog/S102
Accept: application/xml
If-None-Match: "S102-xml-r7"

304 Not Modified
ETag: "S102-xml-r7"
Vary: Accept
```

- A strong ETag for different JSON and XML bytes must not falsely assert byte identity.
- Select the representation before evaluating its conditional response.
- 304 has no response content; the client reuses its valid stored representation.

Takeaway: A business revision number alone is not automatically a valid strong ETag for every variant.

Separate cacheable endpoint example; our classroom API retains no-store.

RFC 9111: caching | RFC 9110: HTTP semantics

Not Every Response Needs Serialization

Case	Required behavior
`HEAD`	Use GET-equivalent selection/metadata, but never send response content.
`204 No Content`	Do not append a JSON, XML, or YAML success body.
`304 Not Modified`	Send validator/cache metadata as required; no response content.
Ordinary `200 / 201`	Encode the selected representation and identify its real media type.

Takeaway: Content negotiation is not permission to add a body to an otherwise bodyless response.

RFC 9110: HTTP semantics

Negotiation Checkpoint: Predict the Outcome

Request	Under our read-endpoint policy
`Accept: application/json`	200 with JSON.
`Accept: application/xml;q=0.5, application/json;q=1`	200 with JSON.
`Accept: application/json;q=0, */*;q=0.7`	XML wins the remaining equal-quality candidates by our preference order.
`Accept: image/png`	406; no supported success representation.
POST with `Content-Type: text/plain`	415 on the JSON-only write operation.

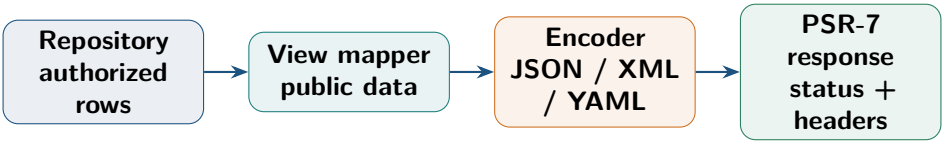
Server preference for ties: JSON, then XML, then YAML. Negotiating a format never bypasses authorization or validation.

RFC 9110, 12.5.1: Accept | OWASP: REST security

3. Implementing the Representation Layer

Keep the query, public view, encoder, and HTTP response separate

Add Encoders, Not Three Different Controllers



The new responsibility

Negotiation selects a supported media type. A representation service encodes the already-selected public view. The emitter sends the final response only after outer middleware returns.

Takeaway: Do not run a different seat-allocation algorithm merely because Accept requested XML.

PHP-FIG: PSR-7 messages | PHP-FIG: PSR-15 middleware

Build One Canonical Public View


```php
function sectionView(array $row): array
{
    return [
        'id' => $row['id'],
        'courseCode' => $row['course_code'],
        'title' => $row['title'],
        'term' => $row['term'],
        'state' => $row['state'],
        'capacity' => (int) $row['capacity'],
        'availableSeats' => (int) $row['capacity']
                          - (int) $row['seats_taken'],
    ];
}
```

Takeaway: Explicit mapping preserves types and hides internal fields. Every encoder receives the same validated view.

Expected row: capacity 30, seats_taken 26, and the Week 5 course JOIN giving title Web Services.

JSON Encoding in PHP


```php
function sectionJson(array $view): string
{
    return json_encode($view,
        JSON_THROW_ON_ERROR
        | JSON_UNESCAPED_UNICODE
        | JSON_UNESCAPED_SLASHES);
}

$body = sectionJson($view); // Produce all bytes before emission.
$media = 'application/json';
```

- Use `JSON_THROW_ON_ERROR` rather than silently returning false or partial output.
- An invalid UTF-8 value or non-finite number indicates a serialization problem.
- Do not apply `JSON_NUMERIC_CHECK` to identifiers that happen to look numeric.

Takeaway: Serialize a public view, not arbitrary PHP objects, resources, or exception instances.

PHP: json_encode

XML Encoding with an Explicit Field Map


```php
function sectionXml(array $view): string {
    $xml = new XMLWriter();
    $xml->openMemory();
    $xml->startDocument('1.0', 'UTF-8');
    $xml->startElement('section');
    foreach (['id', 'courseCode', 'title', 'term', 'state',
              'capacity', 'availableSeats'] as $key) {
        $xml->writeElement($key, (string) $view[$key]);
    }
    $xml->endElement(); $xml->endDocument();
    return $xml->outputMemory();
}
```

Takeaway: XMLWriter escapes text values. Never accept an arbitrary input key as an XML element name.

Requires ext-xmlwriter; fixed non-null Section view only. Collections/nulls need their own mapping.

PHP: XMLWriter

YAML Encoding with a Maintained Library


```php
use Symfony\Component\Yaml\Yaml;

function sectionYaml(array $view): string {
    return Yaml::dump($view, 6, 2,
        Yaml::DUMP_EXCEPTION_ON_INVALID_TYPE
        | Yaml::DUMP_MULTI_LINE_LITERAL_BLOCK);
}

// composer require symfony/yaml:^7.4
// require __DIR__ . '/vendor/autoload.php';
```

- Pass only the public array/map, not an object graph.
- Keep parser/dumper dependencies and lockfile under version control.
- Round-trip the output with the consumer's expected mapping in tests.

Takeaway: Do not write a YAML serializer using string concatenation and indentation guesses.

Symfony 7.4: Yaml component

Use a Negotiation Component, Then Test Its Policy


```php
// Application interface; not an existing framework API.
interface SuccessFormatSelector {
    public function select(?string $accept, array $supported): ?string;
}
$accept = $request->hasHeader('Accept')
    ? $request->getHeaderLine('Accept') : null;
$media = $selector->select($accept,
    ['application/json', 'application/xml', 'application/yaml']);
if ($media === null) {
    throw new ApiProblem(406, 'No acceptable success format.');
}
```

Takeaway: Test q=0 exclusion, specificity, wildcards, malformed input, and ties. A library default need not match the course contract.

Interface/usage sketch: implement the adapter with a maintained parser and the earlier selection algorithm. Bound input, distinguish an absent field from an empty one, and document the empty-field policy.

Negotiation: PHP library | RFC 9110, 12.5.1: Accept

Create a Response Without Echo or Exit

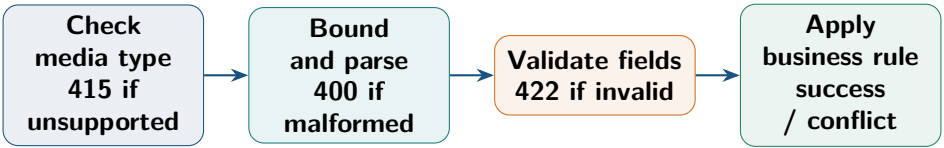

```php
// PSR-17 response factory; mapped $view.
$bytes = match ($media) {
    'application/json' => sectionJson($view),
    'application/xml'  => sectionXml($view),
    'application/yaml' => sectionYaml($view),
};
$type = $media === 'application/xml' ? $media . '; charset=utf-8' : $media;
$response = $factory->createResponse(200)
    ->withHeader('Content-Type', $type)
    ->withHeader('Vary', 'Accept')
    ->withHeader('Cache-Control', 'no-store');
$response->getBody()->write($bytes);
return $response;
```

Takeaway: Return the response to middleware. Emit once, at the outer application boundary.

Fresh response shown. Merge existing Vary tokens; apply HEAD/bodyless policy at emission.

PHP-FIG: PSR-7 messages | PHP-FIG: PSR-15 middleware

Request Decoding Is a Separate Input Contract



- Keep enrollment writes JSON-only until additional formats are deliberately specified.
- Bound request bytes, nesting, collection sizes, and accepted properties.
- A valid XML or YAML document is not automatically valid enrollment data.

Takeaway: One accepted input format is often safer than three poorly specified ones.

OWASP: REST security | PHP: json_decode

JSON Input: Parse, Then Validate


```php
// Run after Content-Type and an 8192-byte body limit are checked.
try {
    $value = json_decode($raw, false, 32, JSON_THROW_ON_ERROR);
} catch (JsonException) {
    throw new ApiProblem(400, 'Request body must be valid UTF-8 JSON.');
}
if (!$value instanceof stdClass) {
    throw new ApiProblem(422, 'The JSON root must be an object.');
}
$fields = get_object_vars($value);
// Now check required properties, allowed fields, types, and business rules.
```

Takeaway: Decoding to an object first distinguishes an empty object from an empty JSON array.

Default json_decode behavior does not reject every duplicate-key input. Require unique keys and use a duplicate-aware parser when strict rejection is part of your policy.

PHP: json_decode | RFC 8259: JSON

XML and YAML Input Need Additional Defenses

⚠ XML input extension

- Disable DTD/external entity features; reject unsupported DOCTYPE input.
- Bound size/depth; use trusted local schemas.
- Blocking network access alone does not prevent local-file entity access.

⚠ YAML input extension

- Do not enable object deserialization or constant resolution.
- Reject/bound aliases and nesting; reject unsupported tags.
- Validate resulting structure and scalar types.

💡 **Takeaway:** Our writes remain JSON-only. These are safeguards to design before adding more request formats.

OWASP: XML external entities | Symfony 7.4: Yaml component | RFC 9512: application/yaml

OpenAPI: Describe Each Success Representation


```yaml
responses:
  '200':
    description: The same section view in a selected format
    headers: {Vary: {schema: {type: string}, example: Accept}}
    content:
      application/json:
        schema: {$ref: '#/components/schemas/Section'}
      application/xml:
        schema: {$ref: '#/components/schemas/Section'}
      application/yaml:
        schema: {$ref: '#/components/schemas/Section'}
```

Takeaway: Use content media-type keys, not a normal header parameter named Accept or Content-Type.

Accept, Content-Type, and Authorization header parameters are ignored by OAS. Use content/security constructs. Fragment shown.

OpenAPI 3.1.2 | Learn OpenAPI: message content

Document XML Mapping and the Error Fallback


```yaml
Section:
  type: object
  xml:
    name: section
  properties:
    id: {type: string}
    capacity: {type: integer}
# Full Section fields remain required.
```

```yaml
'406':
  description: No acceptable success variant
  content:
    application/problem+json:
      schema:
        $ref: '#/components/schemas/Problem'
```

- Add complete examples for every format, including an empty collection and nullable links.
- State that response format choices do not expand accepted request media types.

Takeaway: A schema reused for XML needs an explicit serialization mapping; test actual XML, not only the YAML contract.

OpenAPI 3.1.2 | RFC 9457: Problem Details

Serialization Before Emission Prevents Corrupt Responses

Too late to repair

```
echo '<section>';
// Query or encoder fails here.
echo '{"error":"failed"}';
```

The client receives a partial, mixed-format response.

A safer boundary

```
$view = sectionView($row);
$bytes = sectionXml($view);
// Build and return a response only
// after representation encoding succeeds.
```

Takeaway: Do not print warnings or debugging output into the API response. Once output is sent, a replacement error may be impossible.

PHP: errors and exceptions | PHP-FIG: PSR-15 middleware

4. Error Handling as an API Contract

Communicate a useful outcome without exposing internal failures

Three Different Kinds of Failure

Kind	Registration example	Treatment
Invalid request	Malformed JSON or negative page.	Reject predictably; no partial work.
Expected domain rejection	Student already enrolled; section is full.	Report a defined conflict and preserve state.
Unexpected server failure	Encoder bug or unexpected PDO failure.	Safe 5xx response; internal evidence and recovery.

Takeaway: Do not turn every rejection into 500, and do not hide every failure behind 200.

RFC 9110: HTTP semantics | RFC 9457: Problem Details

Separate Parsing, Validation, and Negotiation

Condition	Our response	Why
Malformed JSON	`400 Bad Request`	The declared input could not be parsed.
Request body too large	`413 Content Too Large`	The input exceeds the configured budget.
XML sent to JSON-only write	`415 Unsupported Media Type`	The operation does not accept that input.
Valid JSON, wrong field type	`422 Unprocessable Content`	Syntax is valid; application input is invalid.
Only PNG accepted for Section	`406 Not Acceptable`	No supported success variant under our policy.

The 400/422 split is a documented API design decision; other APIs may use 400 for validation too.

RFC 9110: HTTP semantics

Status Codes Carry Meaning and Sometimes Headers

Situation	Status / header	Client interpretation
No current authentication	`401` + `WWW-Authenticate`	Authenticate using a supported scheme.
Known caller lacks permission	`403`	Authentication alone is insufficient.
Unknown resource/path	`404`	Target is unavailable or undisclosed.
Known path, wrong method	`405` + `Allow`	Choose an allowed method.

Takeaway: Do not leak resource existence unnecessarily; a server may use 404 to conceal a forbidden target.

Security cases preview later weeks. The local classroom enrollment implementation still has no authentication/ownership enforcement.

RFC 9110: HTTP semantics

Conflicts, Preconditions, and Temporary Failures

Case	Typical status	Useful response
Last seat already taken	`409 Conflict`	Explain the conflict; refresh the state.
If-Match no longer matches	`412 Precondition Failed`	Refresh the view before updating.
Request rate exceeded	`429 Too Many Requests`	Apply a retry policy; inspect Retry-After if supplied.
Temporary service unavailability	`503 Service Unavailable`	A Retry-After value may guide recovery.
Gateway waits too long upstream	`504 Gateway Timeout`	Not a generic label for every slow local operation.

Takeaway: Never retry a write blindly merely because its response is 5xx or a timeout.

RFC 9110: HTTP semantics

Problem Details: A Shared Error Shape


```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json
X-Request-ID: r-7b2c

{
  "type": "https://api.example.edu/problems/section-full",
  "title": "Section is full",
  "status": 409,
  "detail": "Section S101 has no available seats.",
  "instance": "/problems/occurrences/r-7b2c",
  "requestId": "r-7b2c"
}
```

Takeaway: The HTTP status is 409; the body describes that outcome. requestId is our extension, not an RFC-required field.

RFC 9457 replaces RFC 7807. URIs here are illustrative, not live diagnostic endpoints.

RFC 9457: Problem Details

Read the Five Standard Problem Members

Member	What it means	Our rule
`type`	URI identifying a problem category.	Clients can branch on a stable problem type.
`title`	Short description of that category.	Stable except for localization.
`status`	HTTP status reproduced in the content.	Keep it equal to the actual response status.
`detail`	Safe explanation of this occurrence.	Readable by a person; not a parser contract.
`instance`	URI identifying this occurrence.	Need not expose a public stack trace or even be dereferenceable.

Takeaway: RFC 9457 does not require all five members; our API schema deliberately requires them for consistency.

RFC 9457: Problem Details

Field Errors Are Documented Extensions


```
{
  "type": "https://api.example.edu/problems/invalid-enrollment",
  "title": "Invalid enrollment data",
  "status": 422,
  "detail": "Correct the listed fields.",
  "instance": "/problems/occurrences/r-91ac",
  "invalidParams": [
    {"name":"studentId", "reason":"must be a positive integer"},
    {"name":"sectionId", "reason":"is required"}
  ]
}
```

Takeaway: invalidParams is our extension, not a universal RFC property. Describe its schema and stable meaning.

Do not echo submitted passwords, tokens, SQL strings, or private values into validation messages.

RFC 9457: Problem Details | OWASP: REST security

How Do Errors Work When Accept Requests XML?

Our explicit error-format policy

Successful Section reads negotiate JSON, XML, or YAML. Errors use a safe `application/problem+json` fallback, even when that type was not preferred. Clients must inspect the returned Content-Type.

```
Accept: application/xml, application/problem+json
# Success: application/xml
# Conflict or server failure: application/problem+json
```

- This is a documented choice permitted by HTTP, not a universal negotiation rule.
- RFC 9457 also defines `application/problem+xml`; it does not define a standard `application/problem+yaml`.

Takeaway: A 406 handler must not trigger the same failed negotiation again and recurse.

RFC 9457: Problem Details | RFC 9110, 12.5.1: Accept

Map Known Exceptions; Keep Unknown Ones Private

Expected application problem

```
throw new ApiProblem(
    409, 'Section has no available seats.'
);
```

Only explicitly safe messages may pass to the client.

Unexpected Throwable

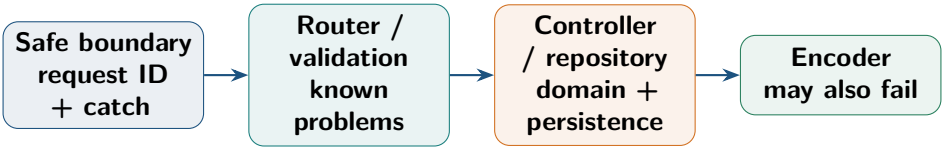
```
catch (Throwable $error) {
    // Record selected internal context.
    // Return a generic 500 problem.
}
```

Never automatically forward `$error->getMessage()` into a public response.

Takeaway: A PDO error may contain SQL or database details. An exception class is not an HTTP response by itself.

RFC 9457: Problem Details | OWASP: REST security

Keep Failure Handling Outside the Work It Protects

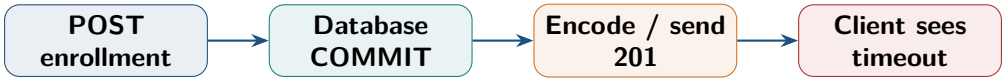


- Protect parsing, validation, application work, and representation encoding.
- On failure, build a fresh response instead of appending an error to partial output.
- Preserve required headers such as Allow or WWW-Authenticate when translating expected errors.

Takeaway: Centralization makes the failure contract consistent; it does not make every exception recoverable.

PHP-FIG: PSR-15 middleware | Slim 4: error middleware

A Transaction Cannot Undo a Lost Response



Committed state can exist even when the client never receives the successful response.

- Validate input and negotiate the response format before an irreversible commit.
- After commit, a serialization or delivery failure does not roll back the committed enrollment.
- Request IDs aid diagnosis; a scoped idempotency-key design addresses safe replay separately.

Takeaway: Keep the Week 4 transaction guarantee and the retry guarantee distinct.

PHP: PDO transactions | RFC 9110: HTTP semantics

Client Recovery Must Match the Operation

Failure	Reasonable next step
422 validation	Correct the input; resending it unchanged is not useful.
406 / 415	Change the accepted response format or declared/sent request format.
409 conflict	Refresh relevant state, resolve the conflict, then decide what to do.
429 / 503	Bound retries and backoff; inspect Retry-After when present.
Timeout after a write	Outcome is uncertain. Reconcile state or use an idempotency contract; do not assume failure.

Takeaway: A server-generated request ID helps support correlate evidence; it is not a replay token.

RFC 9110: HTTP semantics | RFC 9457: Problem Details

PHP Errors Are Not All Throwable Exceptions


```
; Configure the PHP runtime serving the API:
display_errors = Off
log_errors = On
error_reporting = E_ALL

; Keep diagnostic output out of HTTP content.
; Configure a protected destination or stderr via the runtime.
```

- TypeError and many failures are Throwable; ordinary warnings are not all converted automatically.
- A deliberate error handler can promote selected warnings to exceptions; test that policy.
- Parse/startup failures, out-of-memory failures, and broken connections may escape normal recovery.

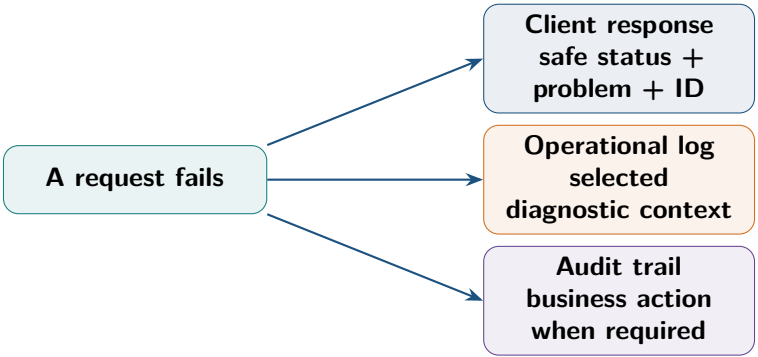
Takeaway: A catch block cannot promise a well-formed response after every possible infrastructure failure.

PHP: errors and exceptions | PHP: error configuration

5. Logging and Operational Evidence

Explain what happened without leaking what should remain private

An Error Has More Than One Audience



Takeaway: The public error body and the private diagnostic record have different purposes and access rules.

RFC 9457: Problem Details | OWASP: logging

Logs, Metrics, and Traces Answer Different Questions

Signal	Question	Registration example
Logs	What happened in this event?	Encoding failed in request r-7b2c.
Metrics	How much, how often, how long?	Failure rate or duration distribution over many requests.
Traces	Where did one distributed operation spend time?	Gateway, registration service, and database spans.

Takeaway: One printed error line is not observability. Correlate useful signals; do not collect every possible payload.

An access record says a response was prepared or sent by a component; it does not prove the client received or understood it.

OpenTelemetry: signals | OWASP: logging

Structured Records Are Easier to Search


```
{
  "message": "http.request.completed",
  "context": {
    "requestId": "r-7b2c", "method": "GET",
    "route": "/v2/sections/{id}", "status": 200,
    "mediaType": "application/xml", "durationMs": 12.4
  },
  "level": 200, "level_name": "INFO", "channel": "registration",
  "datetime": "2026-10-02T14:00:00.000000+00:00",
  "extra": {}
}
```

Takeaway: Use a stable event name and structured context, not a long sentence that must be reverse-engineered.

Illustrative Monolog-style JSON record. Time, duration, and request ID are examples, not measured course traffic. The actual sink writes one record per line.

Monolog 3: usage | Monolog: JSON formatter

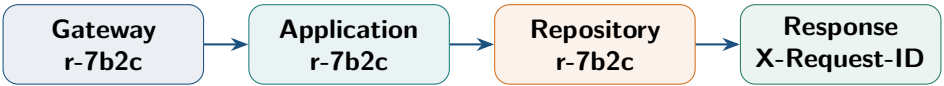
Choose a Small Useful Logging Vocabulary

Field	Why record it?	Caution
Timestamp / duration	Order events; measure elapsed time.	UTC timestamp; monotonic clock for elapsed duration.
Request ID	Join response and internal records.	Generate or validate; never treat as authentication.
Method / route name	Locate the operation.	Prefer templates; raw URLs may contain personal data.
Status / media type	Observe the final outcome.	Record the final response, not a pre-exception guess.
Error category	Diagnose a failure class.	Avoid raw SQL, secrets, and unbounded exception strings.

Takeaway: Record enough to diagnose the operation, not everything available in the request.

OWASP: logging | PHP: hrtime

Follow One Request Across Components




```
http.request.received   requestId=r-7b2c
representation.failed  requestId=r-7b2c  exceptionType=JsonException
http.request.completed requestId=r-7b2c  status=500
```

- The client supplies the response ID to support; staff correlate protected records.
- A later retry is a new HTTP request unless a separate retry-identity policy links it.
- Distributed tracing uses a context such as W3C `traceparent`; do not invent an incompatible trace format.

Takeaway: Correlation is not authentication and not proof that a mutation happened exactly once.

W3C Trace Context | OWASP: logging

PSR-3 Levels: Routine Through Suspicious Events

Level	Meaning	Possible example
`debug`	Detailed diagnostic information.	Bounded encoder choice in a controlled debugging session.
`info`	Routine informative event.	Request completed with selected metadata.
`notice`	Normal but significant event.	A defined business transition or planned migration step.
`warning`	Unusual condition worth attention.	Repeated malformed requests or a deprecated route still in use.

Takeaway: An expected 404 or 409 is not automatically an application error. Choose a severity and volume policy.

PHP-FIG: PSR-3 logging

PSR-3 Levels: Failure and Urgency

Level	Meaning	Possible example
`error`	Runtime failure to investigate.	Unexpected serialization failure for one request.
`critical`	Serious component failure.	An essential registration subsystem is unavailable.
`alert`	Immediate action is needed.	A widespread outage breaches an operational threshold.
`emergency`	The system is unusable.	The whole service platform cannot operate.

Takeaway: HTTP status and log severity are different dimensions. A single 500 should not automatically page every operator.

PHP-FIG: PSR-3 logging

PSR-3 Separates Application Code from the Logger


```php
use Psr\Log\LoggerInterface;

function recordCompletion(LoggerInterface $log, array $context): void
{
    $log->info('http.request.completed', $context);
}

// In a protected diagnostic sink, PSR-3 uses the key "exception":
// $log->error('request.failed', ['exception' => $error]);
```

- LoggerInterface standardizes the methods; it does not choose storage or retention.
- Exception objects can reveal messages, stack frames, or arguments: redact and restrict access before enabling rich exception records.

Takeaway: For the shared classroom log, record an allowlisted error category and request ID instead of raw exceptions.

PHP-FIG: PSR-3 logging | OWASP: logging

Monolog 3: A JSON Stream Logger


```php
use Monolog\Logger;
use Monolog\Level;
use Monolog\Handler\StreamHandler;
use Monolog\Formatter\JsonFormatter;
$handler = new StreamHandler('php://stderr', Level::Info);
$handler->setFormatter(new JsonFormatter(
    JsonFormatter::BATCH_MODE_JSON, true
));
$log = new Logger('registration');
$log->setTimezone(new DateTimeZone('UTC'));
$log->pushHandler($handler);
```

Takeaway: The formatter structures each record; the handler chooses its destination and threshold.

Install Monolog 3 and load vendor/autoload.php. stderr is captured by the hosting/runtime setup; it is not automatically a permanent archive.

Monolog 3: usage | Monolog: JSON formatter

Measure Duration; Record the Final Status


```php
$started = hrtime(true);       // monotonic elapsed-time source
$response = $next->handle($request); // inside a tested error boundary
$durationMs = (hrtime(true) - $started) / 1_000_000;

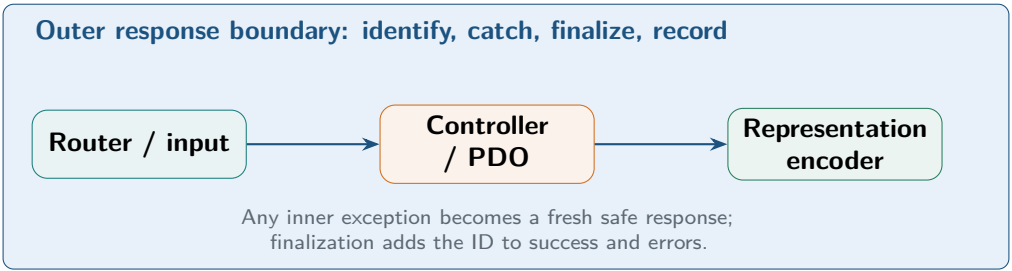
$log->info('http.request.completed', [
    'requestId' => $id,
    'method' => $request->getMethod(),
    'status' => $response->getStatusCode(),
    'mediaType' => $response->getHeaderLine('Content-Type'),
    'durationMs' => round($durationMs, 2),
]);
```

Takeaway: This after-code is skipped when the inner call throws. The boundary must convert exceptions before recording the final response.

A route template can be added through the framework's routing context. Do not assume an immutable downstream request update appears automatically in the outer request.

PHP: hrtime | PHP-FIG: PSR-15 middleware | PHP-FIG: PSR-7 messages

Do Not Let Error Responses Lose Their Request ID



Takeaway: An inner component's after-code cannot decorate an error response created outside it.

PSR-15 recommends an outer exception-to-response layer. This boundary must also guard its formatter/logger failure paths; the emitter remains outside application processing.

PHP-FIG: PSR-15 middleware | Slim 4: error middleware

Boundary Logic: Known Failure, Unknown Failure, Finalization


```php
$id = bin2hex(random_bytes(16));
$request = $request->withAttribute('requestId', $id);
try {
    $response = $next->handle($request);
} catch (ApiProblem $error) {
    $response = $problems->fromExpected($error, $id);
} catch (Throwable $error) {
    $safeLog('error', 'http.request.failed', [
        'requestId' => $id, 'exceptionType' => $error::class,
    ]);
    $response = $problems->unexpected($id); // generic 500
}
$response = $response->withHeader('X-Request-ID', $id);
// Record final status/duration through a guarded logger, then return.
return $response;
```

Integration excerpt: ApiProblem, the problem factory, and safeLog are application services with explicit tests. The boundary uses the fixed JSON error policy, not the failed success serializer.

PHP-FIG: PSR-15 middleware | RFC 9457: Problem Details

A Log Entry Must Not Become a Data Leak

Exclude by default	Safer alternative
Authorization / Cookie / Set-Cookie	A boolean authenticated flag or approved identity category, when justified.
Passwords, API keys, reset tokens	Never record them, including in query strings and exception messages.
Full student records or request bodies	Operation, outcome, correlation ID, and selected non-sensitive fields.
Raw SQL with values and credentials	Query template name and bounded failure category.
Arbitrary incoming request IDs	Generate a local ID or validate trusted boundary metadata.

Takeaway: Use an allowlist of log fields. A generic redactor cannot reliably discover every secret.

OWASP: logging | OWASP: REST security

Logging Has Its Own Input and Output Security

Log injection

```
Untrusted text:
hello\nERROR admin_login_succeeded
```

Hand-built lines could make one event look like two.

Structured handling

A JSON formatter escapes newlines inside string values. Also bound lengths, validate IDs, and encode values when rendering a log viewer.

Takeaway: Escaping protects structure; it does not remove confidential content. Minimize and redact before storage.

OWASP: logging | Monolog: JSON formatter

Access Logs and Audit Trails Are Not the Same

Record	Purpose	Guarantee not implied
HTTP completion log	Diagnose requests, latency, and outcomes.	Does not prove a client received the response.
Business event	Record an enrollment state transition.	Logging before commit does not prove persistence.
Audit trail	Accountability with controlled access and retention.	An ordinary debug file is not an immutable audit system.

Takeaway: Emit business success only after commit. Strict atomic audit delivery may need a transactional outbox or another deliberate design.

OWASP: logging | PHP: PDO transactions

Operate the Logging System Deliberately

- **Destinations:** keep logs outside the public document root; restrict readers and writers.
- **Volume:** rotate/retain according to policy; bound event sizes and use justified sampling.
- **Failure:** test a full disk, a missing sink, and formatter errors; avoid an exception loop.
- **Policy:** ordinary telemetry may degrade gracefully, but required audit failures can demand a different response.

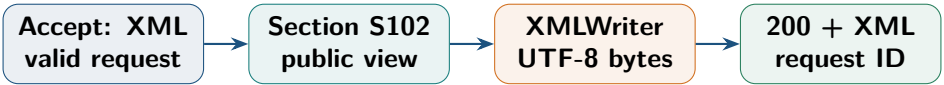
Takeaway: Do not replace a valid response with a leaked logging exception. Define a minimal safe fallback and monitor lost logs.

OWASP: logging

6. Practice and Verification

Compare representation, HTTP outcome, and operational evidence

Worked Journey: The Partner Requests XML



Evidence	Expected observation
Resource / body	S102, Web Services, capacity 30, four available seats.
Response fields	`Content-Type: application/xml; charset=utf-8`; `Vary: Accept`.
Log	One completion record: status 200, media type XML, duration, request ID.
State	No enrollment created; no capacity change.

Takeaway: Representation selection changes how data is delivered, not what a GET is allowed to do.

Worked Failure: A Grading Option Is Invalid


```
PATCH /v1/enrollments/1
Content-Type: application/json
Accept: application/json, application/problem+json

{"gradingOption":"excellent"}
```

```
422 Unprocessable Content
Content-Type: application/problem+json
X-Request-ID: r-91ac

# Safe body explains credit or audit.
# No UPDATE or COMMIT for this input.
```

Takeaway: The JSON parses, but the enum rule fails. Report a correctable client issue, not a PHP stack trace.

Comment lines describe observations; they are not part of an actual Problem Details JSON body.

Worked Failure: The Serializer Crashes

Layer	What must happen
Controller	Obtains the Section view; no bytes have been emitted.
JSON encoder	Rejects an invalid UTF-8 value or another unencodable value.
Error boundary	Creates a safe 500 problem using a minimal reliable fallback.
Response	Correct status and Content-Type; request ID; no raw exception detail.
Logs	Error category plus the same request ID; final completion status 500.

Takeaway: A malformed successful response is not better than a correct error. Investigate the data/encoder cause without leaking it.

PHP: json_encode | RFC 9457: Problem Details

Try the Same Resource with Three Accept Values


```
# Once the Week 6 extension is implemented locally:
curl -i -H "Accept: application/json" \
  "http://127.0.0.1:8000/v2/sections/S102"
curl -i -H "Accept: application/xml" \
  "http://127.0.0.1:8000/v2/sections/S102"
curl -i -H "Accept: application/yaml" \
  "http://127.0.0.1:8000/v2/sections/S102"
```

- Compare status, Content-Type, Vary, identity, values, and request ID.
- Then request `image/png` and inspect the defined 406 response.

These commands target the planned extension, not the unchanged Week 5 server. Bash continuation is shown; use one line or curl.exe in PowerShell.

Test Negotiation and Representation Boundaries

Test	Required evidence
Each supported format	Correct Content-Type; syntactically valid body; same normalized field values.
Absent Accept / wildcard	Documented JSON default and deterministic tie behavior.
Weighted ranges / q=0	Specificity and exclusion respected; no substring matching.
Unsupported success format	406 under our contract; error renderer does not recurse.
XML escaping / empty collections	No markup injection; documented list/null mapping.
HEAD / 204 / 304 cases	No forbidden response content.

Takeaway: A screenshot of pretty XML is not a contract test.

RFC 9110: HTTP semantics | W3C XML 1.0

Test Errors and Logs, Not Only Happy Paths

Test	Expected observation
Malformed input / wrong media	Correct 400/415; no side effects.
Expected conflict / unknown failure	Defined 409 versus safe 500; HTTP and body status agree.
Known sentinel secret submitted	Secret absent from response and approved log sinks.
Newline-containing untrusted field	No forged extra log event; viewer safely encodes it.
Error path and returned rejection	Required request ID and final status available on both.
Logger unavailable	Documented fallback; original response/error is not accidentally replaced.

Takeaway: Test selected diagnostic fields and business state as well as response schemas.

OWASP: logging | RFC 9457: Problem Details

Diagnose the Layer, Not Just the Symptom

Symptom	Inspect first
JSON arrives for an XML request	Accept adapter, supported formats, route scope, and default policy.
XML body says `application/json`	Serializer-to-header mapping or a stale response decorator.
Mixed XML and JSON in one body	Early output; appending errors to partial content; double emission.
JSON variant is served to another client	Vary/cache-key handling and cache eligibility.
500 has no request ID	Exception escaped the finalizer or response was already emitted.
Many duplicate exception records	Multiple logging owners; distinguish access event from diagnostic event.

Takeaway: Follow the path: contract, selection, serializer, response boundary, emitter, and log sink.

Apply the Design to Your Project

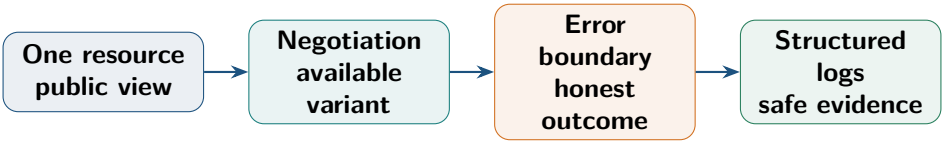
A library catalog

A mobile app uses JSON. A partner requests XML. Staff export YAML. A checkout can fail because a copy was borrowed a moment earlier.

1. Choose the public book/copy/loan representations and cross-format mapping.
2. List accepted request formats separately from response formats.
3. Define defaults, 406/415, and a Problem Details policy.
4. Choose safe log fields and show how one failed checkout is correlated.
5. Specify tests proving equal meaning and no unauthorized data exposure.

Takeaway: The domain changes; the design method and separation of responsibilities remain the same.

The Week 6 Connections to Remember



- Serialization preserves meaning; it does not define authorization or business rules.
- Accept and Content-Type solve different directions of the interaction.
- Errors need stable semantics, correct statuses, and non-recursive rendering.
- Log selected context with a request ID, not raw secrets or uncontrolled exceptions.
- Keep contract, implementation, and tests aligned as the API grows.

7. Reference and Further Reading

Primary specifications, implementation manuals, and focused follow-up

References: Formats and Negotiation

- RFC 9110: HTTP semantics
- RFC 9110, 12.5.1: Accept
- RFC 9111: caching
- RFC 8259: JSON
- W3C XML 1.0
- RFC 7303: XML media types
- YAML 1.2.2
- RFC 9512: application/yaml

Primary-source links checked 2 October 2026. Examples keep the course's pinned conventions rather than claiming every provider uses the same contract.

References: PHP and OpenAPI

- PHP: json_encode
- PHP: json_decode
- PHP: XMLWriter
- Symfony 7.4: Yaml component
- Negotiation: PHP library
- OpenAPI 3.1.2
- Learn OpenAPI: message content

Primary-source links checked 2 October 2026. Examples keep the course's pinned conventions rather than claiming every provider uses the same contract.

References: Error Handling and Logging

- RFC 9457: Problem Details
- PHP-FIG: PSR-7 messages
- PHP-FIG: PSR-15 middleware
- PHP-FIG: PSR-3 logging
- Slim 4: error middleware
- Monolog 3: usage
- Monolog: JSON formatter

Primary-source links checked 2 October 2026. Examples keep the course's pinned conventions rather than claiming every provider uses the same contract.

References: Security and Operations

- OWASP: logging
- OWASP: XML external entities
- OWASP: REST security
- PHP: errors and exceptions
- PHP: error configuration
- W3C Trace Context
- OpenTelemetry: signals

Primary-source links checked 2 October 2026. Examples keep the course's pinned conventions rather than claiming every provider uses the same contract.

Implementation Scope and Figure Attribution

- Continuity: Week 5's eight-record fixture, middleware return path, and paginated collection model.
- Week 6 extends the API design with three success representations and a fixed JSON error fallback; existing code needs adaptation.
- Code slides are focused PHP/PSR integration excerpts, not a complete server or production security implementation.
- All diagrams are original editable TikZ vectors; icons use Font Awesome Free through `fontawesome5`.
- No downloaded images, remote fonts, or external figure files are required to compile this deck.

Takeaway: Validate real library/runtime behavior when assembling the project. A conceptual trace is not an integration test.

Font Awesome Free license